Pemrograman Berorientasi Objek 2026-2027

JOBSHEET W02
CLASS & OBJECT

2.1 Tujuan Praktikum

Setelah melakukan materi praktikum ini, mahasiswa mampu:

1. Mengenal class dan object sebagai konsep dasar pada pemrograman berorientasi objek
2. Mendeklarasikan class beserta atribut dan methodnya
3. Mendeklarasikan constructor
4. Melakukan instansiasi (pembuatan objek baru)
5. Mengakses atribut dan memanggil method dari suatu objek

2.2 Deklarasi Class, Atribut dan Method

Waktu: 40 Menit

Perhatikan Diagram Class berikut ini:

Sepeda
kecepatan: float gear: int
tambahKecepatan(increment:int): void kurangiKecepatan(decrement:int): void cetakInfo(): void

Berdasarkan class diagram di atas, akan dibuat program class dalam Java.

2.2.1 Langkah-langkah Percobaan

1. Buatlah folder baru dengan nama **Praktikum02** kemudian buatlah file baru dengan nama Sepeda.java
2. Lengkapi class **Sepeda** dengan atribut dan method yang telah digambarkan di dalam class diagram di atas

Pemrograman Berorientasi Objek 2026-2027

```java
package Praktikum02;

public class Sepeda {
    float kecepatan;
    int gear;

    public void tambahKecepatan(float increment) {
        kecepatan += increment;
    }

    public void kurangiKecepatan(float decrement) {
        kecepatan -= decrement;
    }

    public void cetakInfo() {
        System.out.println("Kecepatan: " + kecepatan);
        System.out.println("Gear: " + gear);
        System.out.println("====================");
    }
}
```

Haisl Percobaan :

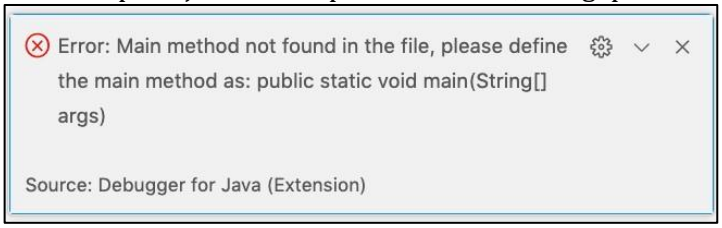
Sepeda.java > Sepeda

```java
public class Sepeda {
    float kecepatan;
    int gear;
    public void tambahKecepatan(float increment) {
    kecepatan += increment;
    }
    public void kurangiKecepatan(float decrement) {
        kecepatan += decrement;
    }
    public void cetakInfo() {
        System.out.println("Kecepatan: " + kecepatan);
        System.out.println("Gear: " + gear);
        System.out.println(x: "====================");
    }
}
```

3. Compile dan run Sepeda.java.

2.2.2 Pertanyaan

1. Ketika Sepeda.java di-compile dan di-run, mengapa error berikut muncul?



Pemrograman Berorientasi Objek 2026-2027

hanya berisi deklarasi class/blueprint dan tidak memiliki titik awal eksekusi main

2. Ada berapa atribut yang dimiliki class Sepeda? Pada baris berapa dilakukan deklarasi atribut?
   - 2 yakni: kecepatan bertipe data float, dan gear bertipe data int
3. Sebutkan method yang dimiliki oleh class Sepeda
   - tambahKecepatan, kurangKecepatan, cetakInfo
4. Sebutkan parameter dari method tambahKecepatan()
   Increment bertipe data float
5. Mengapa method **tambahKecepatan()** memerlukan parameter **increment**?
   Untuk menentukan besarnya nilai penambahan kecepatan yang diinginkan
6. Mengapa method **tambahKecepatan()** tidak memerlukan parameter **kecepatanAwal**?
   Nilai kecepatan yang akan ditambah sudah tersimpan di dalam atribut instansi
7. Mengapa method **cetakInfo()** memiliki return type void?
   Method tersebut hanya bertugas menampilkan/mencetak informasi ke layar
8. Modifikasi method **kurangiKecepatan()** sehingga kecepatan minimum adalah 0

```
    }
    public void kurangiKecepatan(float decrement) {
        kecepatan -= decrement;
        if (kecepatan < 0) {
            kecepatan = 0;
        }
    }
```

9. Modifikasi method **tambahKecepatan()** sehingga kecepatan maksimum adalah 20

```
    int gear;
    public void tambahKecepatan(float increment) {
        kecepatan += increment;
        if (kecepatan > 20) {
            kecepatan = 20;
        }
    }
```

2.3 Instansiasi Objek dan Mengakses Atribut & Method

Waktu: 40 Menit

Class Sepeda telah dibuat sebagai template/cetakan untuk membuat objek-objek bertipe sepeda. Untuk membuat objek baru, perlu dilakukan instansiasi objek.

1. Buatlah class baru dengan nama **SepedaMain** beserta method **main()**.

2. Di dalam method **main()**, lakukan instansiasi objek bernama **sepeda1** dan **sepeda2** kemudian cobalah untuk memodifikasi atribut dan memanggil method.

```java
package Praktikum02;

public class SepedaMain {
    Run | Debug
    public static void main(String[] args) {
        Sepeda sepeda1 = new Sepeda();
        sepeda1.kecepatan = 5;
        sepeda1.gear = 1;
        sepeda1.tambahKecepatan(3);
        sepeda1.cetakInfo();

        Sepeda sepeda2 = new Sepeda();
        sepeda2.cetakInfo();
    }
}
```

3. Run class **SepedaMain** tersebut dan amati hasilnya.

```java
public class SepedaMain {
    Run | Debug
    public static void main(String[] args) {
        Sepeda sepeda1 = new Sepeda();
        sepeda1.kecepatan = 5;
        sepeda1.gear = 1;
        sepeda1.tambahKecepatan(increment: 3);
        sepeda1.cetakInfo();

        Sepeda sepeda2 = new Sepeda();
        sepeda2.cetakInfo();
    }
}
```

```
Kecepatan: 20.0
Gear: 1
====================
Kecepatan: 0.0
Gear: 0
====================
```

2.3.1 Pertanyaan

1. Pada class **SepedaMain**, pada baris berapa dilakukan instansiasi? Apa nama objek yang dihasilkan?

   - baris ketiga : Sepeda sepeda1 = new Sepeda(); hasl objek yang dihasilkan sepeda1

   Baris ke 9 : Sepeda sepeda2 = new Sepeda(); hasl objek yang dihasilkan sepeda2

2. Jelaskan perbedaan class dan object

Merupakan *blueprint*, cetakan, atau *template* yang mendefinisikan struktur, sedangkan object adalah merupakan wujud nyata (*instance*) dari suatu class yang dibuat di memori.

3. Bagaimana cara mengakses atribut dan memanggil method dari suatu objek?

   Mengakses atribut maupun memanggil method dari suatu objek dilakukan dengan menggunakan **operator titik**

4. Bagaimana hasil **cetakInfo()** untuk objek **sepeda2**? Apa kesimpulannya?

   - hanya muncul kecepatan 0 dan gear 5 karena masih belum ada nilai yang di setkan

5. Pada class **Sepeda** tidak terdapat constructor **Sepeda()** secara eksplisit, mengapa objek sepeda tetap dapat diinstansiasi?

   - otomatis menyediakan default constructor

2.4 Constructor

Waktu: 40 Menit

Di dalam percobaan ini, kita akan mempraktekkan bagaimana membuat berbagai macam konstruktor berdasarkan parameternya.

2.4.1 Langkah-langkah Percobaan

1. Pada class **Sepeda**, deklarasikanlah constructor berparameter sebagai berikut

```java
package Praktikum02;

public class Sepeda {
    float kecepatan;
    int gear;

    public Sepeda(float newKecepatan, int newGear) {
        kecepatan = newKecepatan;
        gear = newGear;
    }

    public void tambahKecepatan(float increment) {
        kecepatan += increment;
    }

    public void kurangiKecepatan(float decrement) {
        kecepatan -= decrement;
    }

    public void cetakInfo() {
        System.out.println("Kecepatan: " + kecepatan);
        System.out.println("Gear: " + gear);
        System.out.println("====================");
    }
}
```

2. Run kembali class **SepedaMain** dan amati hasilnya.

```java
public class Sepeda {
    float kecepatan;
    int gear;

    public Sepeda(float newKecepatan, int newGear) {
        kecepatan = newKecepatan;
        gear = newGear;
    }

    public void tambahKecepatan(float increment) {
    kecepatan += increment;
    if (kecepatan > 20) {
        kecepatan = 20;
    }
}
   public void kurangiKecepatan(float decrement) {
    kecepatan -= decrement;
    if (kecepatan < 0) {
        kecepatan = 0;
    }
}
    public void cetakInfo() {
        System.out.println("Kecepatan: " + kecepatan);
        System.out.println("Gear: " + gear);
        System.out.println("====================");
    }
}
```

2.4.2 Pertanyaan

1. Apakah constructor juga merupakan method? Jika iya, apa perbedaan constructor dengan method lainnya?

   - ya, constructor adalah method khusus yang memiliki nama sama persis dengan nama kelas dan tidak mempunyai tipe kembalian (*return type*), bahkan tanpa menggunakan kata kunci

2. Apa yang sebenarnya dilakukan ketika constructor dipanggil?

   sistem akan mengalokasikan ruang memori di *heap* untuk membuat objek baru, lalu menjalankan blok kode di dalam constructor guna menginisialisasi atau memberikan nilai awal pada atribut-atribut objek tersebut sebelum akhirnya mengembalikan alamat referensi objek tersebut.

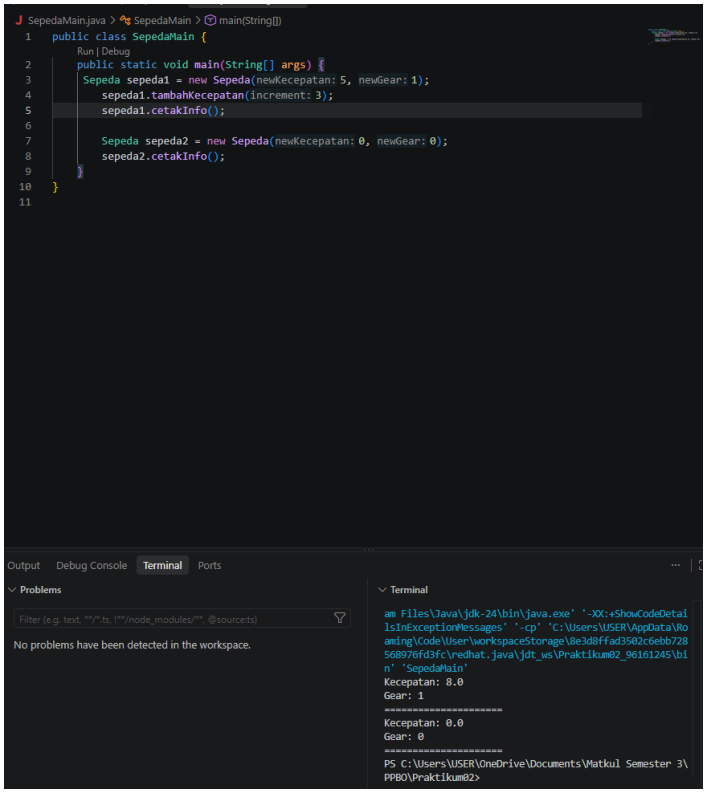
3. Apakah SepedaMain dapat di-run? Mengapa?

   - Tidak dapat di-run (error). ketika constructor berparameter dibuat secara eksplisit, Java tidak menyediakan *default constructor* tanpa parameter secara otomatis, sehingga pemanggilan tanpa argumen menjadi tidak valid.

   Modifikasi SepedaMain sebagai berikut

```java
public class SepedaMain {
    Run | Debug
    public static void main(String[] args) {
        Sepeda sepeda1 = new Sepeda(5, 1);
        sepeda1.tambahKecepatan(3);
        sepeda1.cetakInfo();
    }
}
```

4. Perhatikan bahwa **SepedaMain** sudah dapat di run



5. Suatu class dapat memiliki lebih dari 1 constructor, tambahkan constructor tanpa parameter pada class Sepeda

```
public Sepeda(){

}

public Sepeda(float newKecepatan, int newGear) {
    kecepatan = newKecepatan;
    gear = newGear;
}
```

```
    public Sepeda() {
}

public Sepeda(float newKecepatan, int newGear) {
    kecepatan = newKecepatan;
    gear = newGear;
}
```

6. Modifikasi class SepedaMain sebagai berikut

```
public class SepedaMain {
    Run | Debug
    public static void main(String[] args) {
        Sepeda sepeda1 = new Sepeda(5, 1);
        sepeda1.tambahKecepatan(3);
        sepeda1.cetakInfo();

        Sepeda sepeda2 = new Sepeda();
        sepeda2.kecepatan = 7;
        sepeda2.gear = 1;
        sepeda2.cetakInfo();
    }
}
```

```
        Sepeda sepeda2 = new Sepeda();
        sepeda2.kecepatan = 7;
        sepeda2.gear = 1;
        sepeda2.cetakInfo();
    }
}
```

7. Run SepedaMain dan amati hasilnya. Object sepeda1 dibuat dengan constructor yang mana? Bagaimana dengan object sepeda2? Buatlah kesimpulan terkait constructor.

```
Kecepatan: 8.0
Gear: 1
=====================
Kecepatan: 7.0
Gear: 1
=====================
```

Object sepeda2 tetap berjalan karena menggunakan default construktor

2.5 Tugas

Waktu: 100 menit

Program Game Dragon sederhana

Dragon
x: int y: int direction: int
changeDirection(newDirection: int): void move(steps: int): void printStatus(): void

- Buatlah class Dragon sesuai class diagram di atas

Pemrograman Berorientasi Objek 2026-2027

- Atribut **x** digunakan untuk menyimpan posisi koordinat x (mendatar) dari snake, sedangkan atribut **y** untuk posisi koordinat y (vertikal)
- Atribut direction digunakan untuk menyimpan arah dragon:
  1: atas
  2: kanan
  3: bawah
  4: kiri
- Method **changeDirection()** digunakan untuk mengubah arah dragon berdasarkan parameter newDirection. Implementasikan method changeDirection sedemikian rupa sehingga atribut direction hanya dapat bernilai 1, 2, 3, atau 4.
- Method **move()** digunakan untuk mengubah posisi dragon dengan jumlah langkah sesuai parameter steps. Posisi akan berubah bergantung dengan direction saat ini. Misalnya, jika direction bernilai 1 maka dragon akan berpindah ke arah atas, dan seterusnya.
- Method **printStatus()** digunakan untuk mencetak koordinat dan arah objek dragon
- Buatlah class **DragonMain** kemudian lakukan instansiasi 2 buah objek dari class Dragon. Cobalah melakukan perubahan posisi untuk kedua objek tersebut.
- Apa yang terjadi jika method move() dipanggil persis setelah objek diinstansiasi? Ke mana objek berpindah? Perbaiki kode program untuk mengatasi masalah tersebut.
- Jika move() dipanggil langsung setelah objek diinstansiasi, objek tidak berpindah karena direction masih bernilai 0 sehingga keluar pesan *"Arah belum ditentukan!"*;

Class Dragon :

Dragon.java

```
1   public class Dragon {
2       int x;
3       int y;
4       int direction;
5
6   public void changeDirection(int newDirection) {
7       if (newDirection >= 1 && newDirection <= 4) {
8           direction = newDirection;
9       } else {
10  System.out.println("Arah tidak valid!"); } }
11
12  public void move(int steps) {
13      switch (direction) {
14          case 1: y += steps; break; //atas
15          case 2: x += steps; break; //kanan
16          case 3: y -= steps; break; //bawah
17          case 4: x -= steps; break; //kiri
18      default: System.out.println("Arah belum ditentukan!"); } }
19
20      public void printStatus() {
21          System.out.println("Posisi: (" + x + "," + y + ")");
22          System.out.println("Arah: " + direction);
23          System.out.println("=========="); } }
```

Pemrograman Berorientasi Objek 2026-2027

Class MainDragon :

```
1   public class DragonMain {
        Run | Debug
2       public static void main(String[] args) {
3           Dragon d1 = new Dragon();
4           d1.changeDirection(newDirection: 4);
5           d1.move(steps: 5);
6           d1.printStatus();
7
8           Dragon d2 = new Dragon();
9           d2.changeDirection(newDirection: 13);
10          d2.move(steps: 3);
11          d2.printStatus();
12      }
13  }
14
```

Output :

```
n' 'DragonMain'
Posisi: (-5,0)
Arah: 4
==========
Arah tidak valid!
Arah belum ditentukan!
Posisi: (0,0)
Arah: 0
==========
```